UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Instituto de Investigaciones en Matemáticas Aplicadas y Sistemas

Práctica 1 - Preprocesamiento de datos y extracción de características

PRESENTA

Munive Ramírez Ibrahim

PROFESORA

Dra. María Elena Martínez Pérez

ASIGNATURA

Reconocimiento de Patrones

1. Objetivos

α_1. Identificar y analizar la presencia de correlaciones positivas entre las varibales de un conjunto de datos.

α_2. Aprender a extraer características de imágenes utilizando técnicas de procesamiento de imágenes.

α_3. Observar de manera experimental los efectos del procesamiento de datos, imágenes y señales.

α_4. Familiarizarse con el proceso de limpieza y preprocesamiento de los datos.

α_5. Aprender a generar señales sintéticas y visualizarlas para su análisis o procesamiento posterior.

2. Introducción

2.1. Preprocesamiento y limpieza de datos para el Reconocimiento de Patrones

El preprocesamiento de datos es una etapa fundamental en el reconocimiento de patrones, ya que garantiza la calidad y relevancia de los datos antes de su análisis. Este proceso incluye varias técnicas como limpieza, transformación, reducción y normalización de datos. La limpieza de datos implica la eliminación de valores atípicos, datos incompletos o inconsistentes, mientras que la tranformación convierte los datos en un formato adecuado, como la codificación de variables categóricas o la aplicación de transformaciones matemáticas. La normalización y estandarización aseguran que los datos tengan una escala uniforme, lo cual es crucial para algoritmos que dependen de medidas de distancia. Además, en el caso de datos textuales o de imágenes, se aplican técnicas específicas como la tokenización o el filtrado de ruido. Un buen preprocesamiento no solo mejora la precisión de los modelos de reconocimiento de patrones, sino que también reduce el tiempo de procesamiento y evita problemas de sobreajuste.

En este proceso se aplican diferentes técnicas de análisis de datos que van desde la visión por computadora hasta el procesamiento de señales biomédicas. Las principales técnicas utilizadas en el preprocesamiento de datos son:

- Limpieza de datos. Consiste en eliminar errores, valores atípicos y datos faltantes para asegurar la calidad de los datos antes de su uso en modelos de reconocimiento de patrones. Las principales funciones son: el manejo de valores faltantes, eliminación de outliers y corrección de inconsistencias.
- Normalización y estandarización. Este conjunto de funciones aseguran que las características tengan escalas comparables, mejorando la estabilidad y rendimiento de los algoritmos de reconocimiento de patrones, entre los principales métodos se encuentran la normalización (Min-Max Scaling), estandarización y escalamiento.

- Transformaciones de datos. Mejora la representación de los datos para aumentar la capacidad predictiva y mejorar la separación entre clases. Estas transformaciones pueden ir desde transformación logarítmica y raíz cuadrada, hasta transformadas de Fourier y Wavelet, así mismo se suele utilizar una codificación One-Hot que convierte variables categóricas en vectores binarios y la codificación de frecuencia que asigna valores numéricos en función de la frecuencia de aparición de una categoría.

- Filtrado y eliminación de ruido. Se aplica principalmente en señales e imágenes para mejorar la calidad de los datos y facilitar su análisis. Algunos de los filtros que se utilizan son: filtros de suavizado, filtro Notch, filtro pasa bajas y pasa altas.

- Integración y formateo de datos. Cuando se utilizan múltiples fuentes de datos, es escencial integrarlas y formatearlas de manera coherente. Algunas de las técnicas que podemos utilizar para esto son: la conversión de fotmatos que transforma los tipos de datos para compatibilidad con diferentes herramientas de análisis y la reducción de redundancia que utilizamos para la eliminación de información duplicada para evitar sesgos en el análisis.

2.2. ECG (electrocardiograma)

La señal electrocardiográfica (ECG) es un registro de la actividad eléctrica del corazón y se utiliza ampliamente en el diagnóstico de enfermedades cardiovasculares. El ECG está compuesto por diversas ondas, incluyendo la onda P, el complejo QRS y la onda T, que reflejan la despolarización y repolarización del corazón. Existen diversos métodos para el análisis de la señal ECG, como la descomposición en Fourier, que permite separar sus componentes y mejorar la interpretación de los datos. Sin embargo, las señales ECG pueden estar contaminadas con ruido, como interferencias de línea de alimentación y artefactos de movimiento, lo que requiere técnicas de filtrado para mejorar la calidad del análisis. Además, el desarrollo de algoritmos de clasificación, ha permitido mejorar la detección de patologías cardíacas a partir de la señal ECG.

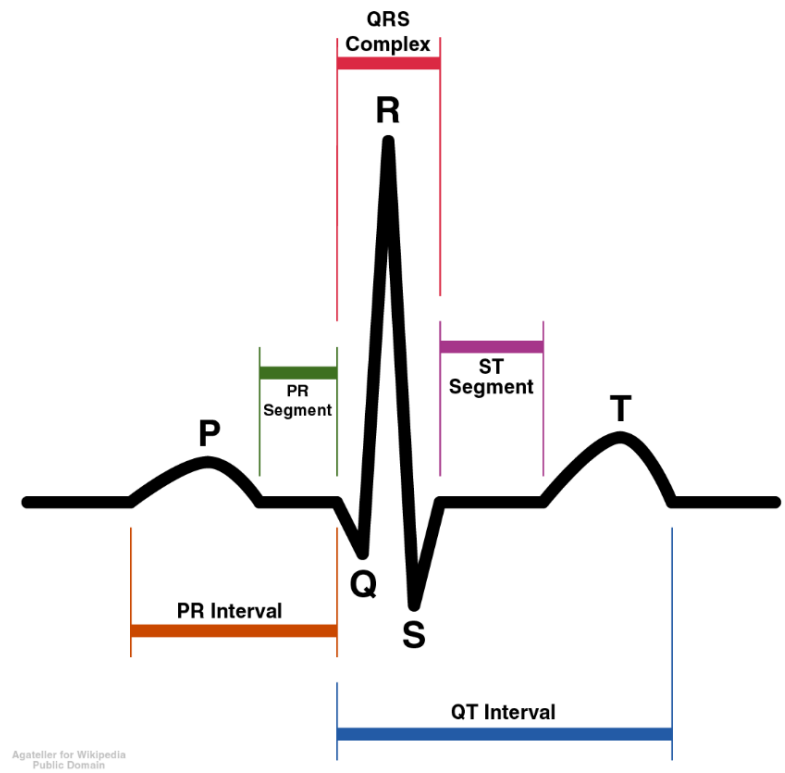

Figura 1: Representación esquemática de un ECG normal.

3. Ejercicios

3.1. Análisis de correlación en el conjunto de datos de la flor Iris.

1. Carga el conjunto de datos de la flor Iris (disponible en MATLAB).
2. Selecciona las cuatro variables numéricas:
   - Longitud del sépalo (sepal length)
   - Ancho del sépalo (sepal width)
   - Longitud del pétalo (petal length)
   - Ancho del pétalo (petal width)
3. Calcula la matriz de correlación entre estas variables utilizando un método adecuado (por ejemplo, el coeficiente de correlación de Pearson).
4. Identifica pares de variables que presenten una correlación positiva significativa (valores cercanos a 1).

5. Visualiza estas correlaciones utilizando un mapa de calor (heatmap) o gráficos de dispersión (scatter plot).
6. Interpreta los resultados:
   - ¿Qué pares de variables muestran una correlación positiva fuerte?
   - ¿Cómo podrían estas correlaciones estar relacionadas con las características de las diferentes especies de Iris?
7. (Opcional) Repite el análisis separando los datos por especie (setosa, versicolor, virginica) para ver si las correlaciones varían entre ellas.

3.2. Extracción de Características en Imágenes.

1. Carga una imagen de tu elección (Asegurarse tener instalado el Image Processing Toolbox de MATLAB).
2. Preprocesamiento de la imagen: Convierte la imagen a escala de grises si es necesario. Aplica un filtro de suavizado (por ejemplo, un filtro Gaussiano) para reducir el ruido.
3. Extracción de bordes: Utiliza el operador de Prewitt o Sobel para detectar bordes en la imagen.
4. Descriptores de forma: Utiliza la transformada de Hough para detectar líneas o círculos en la imagen. Extrae propiedades de regiones utilizando `regionprops`.
5. Visualizar y comparar los resultados:
   - Visualiza los resultados de cada paso (imagen suavizada, bordes detectados, círculos detectados, etc.).
   - Analiza las características extraídas (formas, bordes) y su relación con el contenido de la imagen.

3.3. Generación de Señales Sintéticas de ECG.

1. Generar una Señal Sintética de ECG: Utilizar la función ECG de MATLAB o generar una onda sintética con sumas de senoidales. Una forma común es combinar ondas sinusoidales y pulsos para representar las ondas P, QRS y T de un ECG.
2. Añadir ruido a la señal de ECG: Simular ruido de alta frecuencia (por ejemplo, ruido de 60 Hz de interferencia eléctrica) y ruido aleatorio (ruido gaussiano).
3. Transformada de Fourier para analizar el espectro de frecuencia: Utilizar la Transformada Rápida de Fourier (FFT) para analizar las componentes de frecuencia de la señal con ruido.
4. Diseñar un filtro en el dominio de la frecuencia: Crea un filtro para eliminar el ruido de alta frecuencia (por ejemplo, frecuencias superiores a 40 Hz).

5. Comparar la señal original, la señal con ruido y la señal filtrada, superponer las tres señales para observar el efecto del filtrado y comentar:
   - Descripción de los pasos realizados.
   - Resultados obtenidos (gráficos y análisis).
   - Discusión sobre la efectividad del filtrado en la eliminación del ruido.
6. (Opcional) Diseñar un filtro en el dominio del tiempo: Crea un filtro para eliminar el ruido de alta frecuencia y compara los resultados.

4. Código

4.1. Dataset flor Iris

Empezaremos importando las bibliotecas necesarias para realizar esta sección. La parte escencial es saber dónde está el dataset Iris y cargarlo de forma correcta.

```python
# Bibliotecas

import cv2
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from IPython.display import display
```

Listing 1: Carga de bibliotecas.

4.1.1. Carga de flor Iris

Ya con las bibliotecas listas, pasamos a cargar el conjunto iris:

```python
from sklearn.datasets import load_iris

iris = load_iris()

df_iris = pd.DataFrame(data = iris.data, columns = iris.feature_names)
df_iris['species'] = iris.target
df_iris.sample(5)
```

Listing 2: Carga del conjunto Iris.

En esta parte no se hizo nada nuevo o complicado, solo se cargó el dataset, se escogieron las columnas y se imprimieron una muestra de los datos.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
36	5.5	3.5	1.3	0.2	0
13	4.3	3.0	1.1	0.1	0
71	6.1	2.8	4.0	1.3	1
138	6.0	3.0	4.8	1.8	2
144	6.7	3.3	5.7	2.5	2

Cuadro 1: Muestra de datos (Iris dataset).

4.1.2. Variables numéricas

Ahora filtraremos las variables numéricas y haremos una copia de ellas en un nuevo datafrema para no modificar el archivo original.

```
df_variables = df_iris.iloc[:, 0:4]
```

Listing 3: Carga de los variables numéricas.

Ahora, ya con las variables numéricas, calcularemos la matriz de correlación especificando que lo haremos con Pearson.

4.1.3. Matriz de correlación

```
matriz_correlacion = df_variables.corr(method = 'pearson')
```

Listing 4: Matriz de correlación.

4.1.4. Correlaciones positivas

Pasemos con el filtro de correlaciones positivas. Para esta parte utilizaremos una función que nos permita tomar los elementos de una matriz que superen cierto umbral, para ello daremos un parámetro que será nuestro umbral e iteraremos elemento por elemento de la matriz, a excepción de la parte triangular inferior, pues la matriz de correlación es cuadrada y simétrica, es decir, solo se gastarían recursos filtrando la matriz completa.

Una vez recorrida la matriz, devolverá una lista con las correlaciones que superen el umbral indicado.

```
def filtro(matriz, umbral = 0.8):

    variables = matriz.columns
    corre_altas = []

    for i in range(matriz.shape[0]):
```

```python

        for j in range(i + 1, matriz.shape[1]):

            if matriz.iloc[i, j] >= umbral:

                corre_altas.append((variables[i], variables[j], matriz.iloc[i, j])
    )

    return corre_altas
```

Listing 5: Función filtro.

```python
filtro(matriz_correlacion)
```

Listing 6: Uso del filtro.

Al usar esta función obtenemos lo siguientes resultados:

```
[('sepal length (cm)', 'petal length (cm)', np.float64(0.872)),
 ('sepal length (cm)', 'petal width (cm)', np.float64(0.818)),
 ('petal length (cm)', 'petal width (cm)', np.float64(0.963))]
```

Listing 7: Correlaciones positivas altas.

4.1.5. Heatmap de la matriz de correlación

Con ayuda de la biblioteca `seaborn` mostraremos la matriz de correlación con un formato más vistoso.

```python
plt.figure(figsize = (4, 4))

sns.heatmap(matriz_correlacion, annot = True, cmap = 'coolwarm', vmin = -1, vmax =
    1, cbar = False)
plt.title('Matriz de Correlación')
plt.show()
```

Listing 8: Parámetros e impresión del Heatmap.

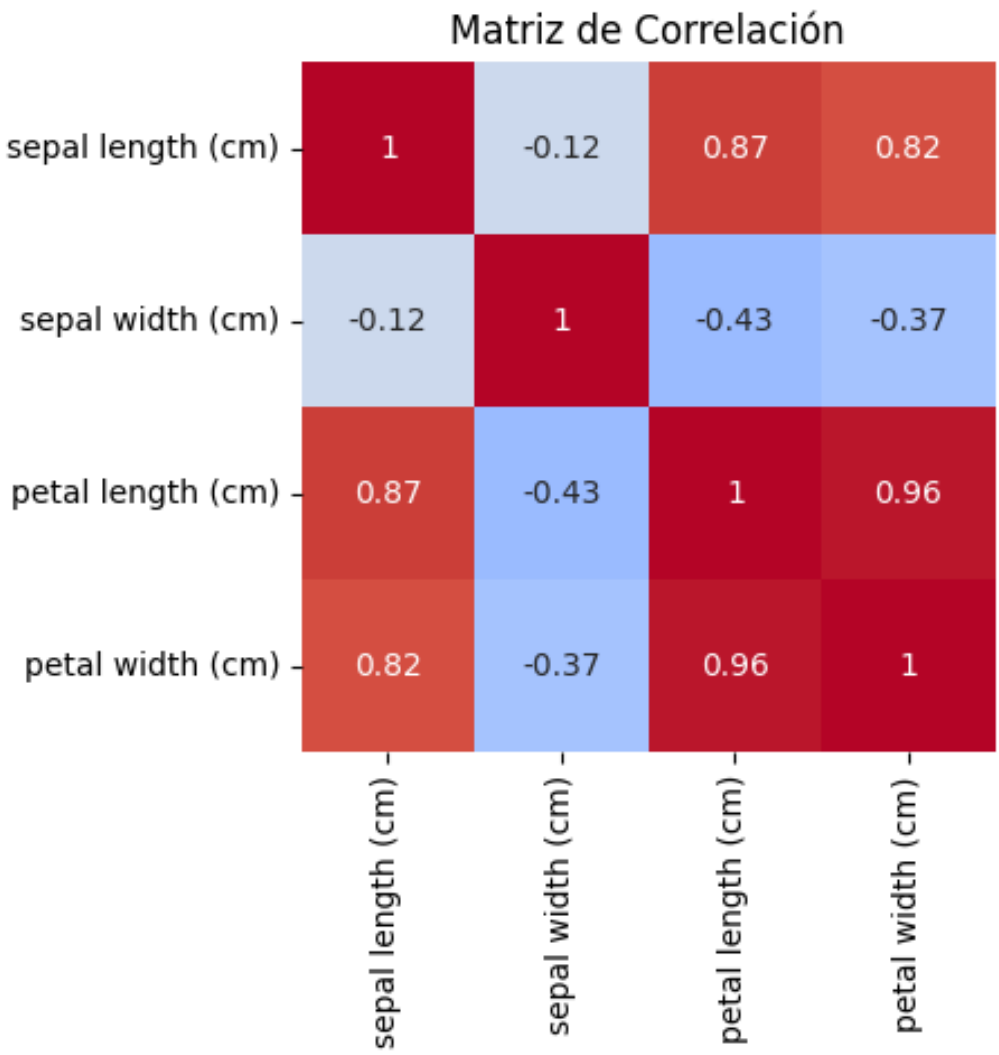

Figura 2: Heatmap de la matriz de Correlación.

4.1.6. Pares de variables con correlación positiva fuerte

Gracias a la función `filtro`, pudimos encontrar las variables con correlación fuerte, las cuales son las siguientes:

- Longitud de sépalo $\Leftrightarrow$ Longitud de pétalo ($\rho = 0{,}871$).
- Longitud de sépalo $\Leftrightarrow$ Ancho de pétalo ($\rho = 0{,}817$).
- Longitud de pétalo $\Leftrightarrow$ Ancho de pétalo ($\rho = 0{,}962$).

4.1.7. ¿Cómo se relacionan las correlaciones con las diferntes especies?

Hasta no hacer el análisis correspondiente, no se puede dar una respuesta sólida; solo una hipótesis de que la correlación de las distintas medidas de las flores definen las especies.

■

4.1.8. Repetir el análisis anterior por especie

Para no repetir el proceso tres veces se optó por hacer un diccionario filtrando las especies, para después iterar y hacer el respectivo análisis.

Definamos el siguiente diccionario:

```
dfs = {especie: datos.iloc[:, 0:4] for especie, datos in df_iris.groupby('species')}
```

Listing 9: Diccionario de dataframes.

Procedemos a calcular las matris de correlación y a gaurdarlas en una lista.

```
matrices_correlacion = []

for i in range(len(dfs)):

    matrices_correlacion.append(dfs[i].corr(method = 'pearson'))
```

Listing 10: .

Ahora visualizamos las matrices de correlación de las distintas especies:

```
for i in range(len(dfs)):

    print(f"Matriz de Correlación de la especie {i}")

    display(matrices_correlacion[i].round(3))
    print("\n")
```

Listing 11: .

Matriz de correlación de la especie 0:

$$\mathcal{M}_{\rho_0} = \begin{bmatrix} 1 & 0{,}743 & 0{,}267 & 0{,}278 \\ 0{,}743 & 1 & 0{,}178 & 0{,}233 \\ 0{,}267 & 0{,}178 & 1 & 0{,}332 \\ 0{,}278 & 0{,}233 & 0{,}332 & 1 \end{bmatrix}$$

Matriz de correlación de la especie 1:

$$\mathcal{M}_{\rho_1} = \begin{bmatrix} 1 & 0{,}526 & 0{,}754 & 0{,}546 \\ 0{,}526 & 1 & 0{,}561 & 0{,}664 \\ 0{,}754 & 0{,}561 & 1 & 0{,}787 \\ 0{,}546 & 0{,}664 & 0{,}787 & 1 \end{bmatrix}$$

Matriz de correlación de la especie 2:

$$\mathcal{M}_{\rho_2} = \begin{bmatrix} 1 & 0{,}457 & 0{,}864 & 0{,}281 \\ 0{,}457 & 1 & 0{,}401 & 0{,}538 \\ 0{,}864 & 0{,}401 & 1 & 0{,}322 \\ 0{,}281 & 0{,}538 & 0{,}322 & 1 \end{bmatrix}$$

Ahora aplicaremos la función `filtro` para encontrar las correlaciones altas de cada una de las matrices.

```
for i in range(len(dfs)):

    print(f'\nRelaciones con correlación alta de la especie {i}:')
    print(filtro(matrices_correlacion[i].round(3), 0.75))
```

Listing 12: Correlaciones altas de las matrices.

```
Relaciones con correlación alta de la especie 0:
[]

Relaciones con correlación alta de la especie 1:
[('sepal length (cm)', 'petal length (cm)', np.float64(0.754)), ('petal length (cm
    )', 'petal width (cm)', np.float64(0.787))]

Relaciones con correlación alta de la especie 2:
[('sepal length (cm)', 'petal length (cm)', np.float64(0.864))]
```

Listing 13: Relaciones con correlación alta.

Finalmente imprimimos cada uno de los heatmaps.

```
plt.figure(figsize = (18, 6))

colores = ['Reds', 'Greens', 'Blues']

for i, (matriz, color) in enumerate(zip(matrices_correlacion, colores)):

    plt.subplot(1, 3, i + 1)

    sns.heatmap(
        matriz,
        annot = True,
```

```
        cmap = color,
        vmin = -1,
        vmax = 1,
        cbar = False
    )

    plt.title(f'Matriz de correlación de la especie {i}')

plt.tight_layout()
plt.show()
```

Listing 14: Heatmaps por especie.

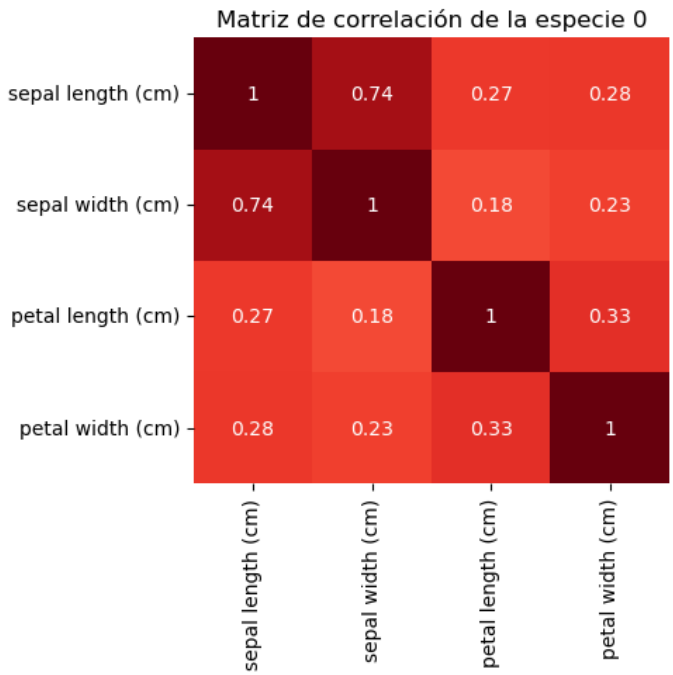

Figura 3: Heatmap de la especie 0.

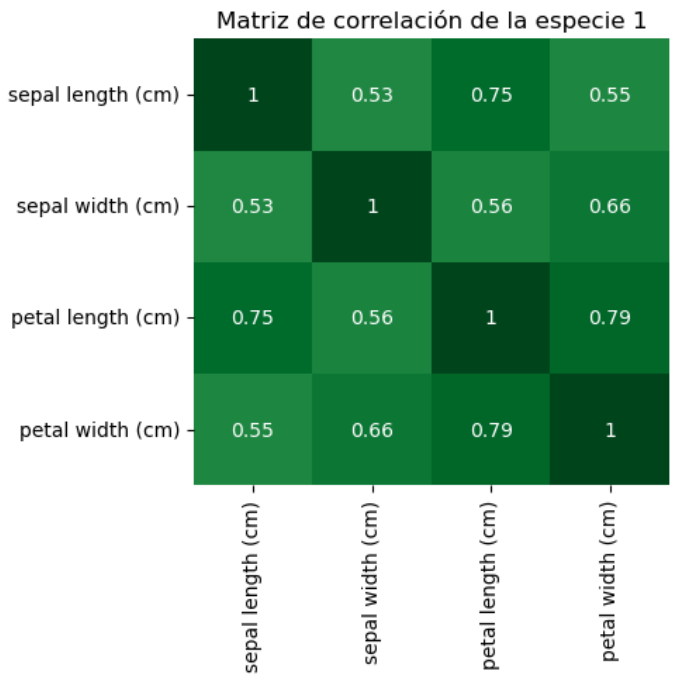

Figura 4: Heatmap de la especie 1.

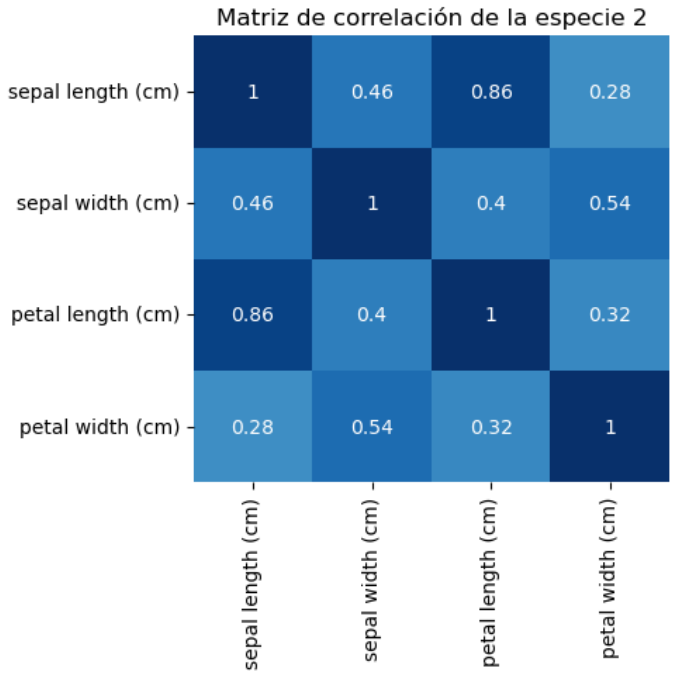

Figura 5: Heatmap de la especie 2.

4.2. Características de imágenes

Similar al ejercicio pasado, empezaremos cargando las bibliotecas necesarias para realizar los ejercicios.

```
# Bibliotecas

import cv2
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from skimage.measure import regionprops
from skimage.measure import label as sk_label
```

Listing 15: Carga de bibliotecas

4.2.1. Carga de imágenes

Proporcionaremos la dirección y el nombre del archivo para poder trabajar con la imagen y usaremos `cv2` para poder leer y trabajar con la siguiente imagen:

Figura 6: Imagen a trabajar.

```
img = cv2.imread('dune.jpeg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Listing 16: Carga de la imagen a trabajar.

4.2.2. Filtros de imágenes

Ya con nuestra imagen cargada, buscamos eliminar el mayor ruido posible para que se identifiquen de mejor los bordes, líneas y figuras.

¿Cómo sé cuál filtro es mejor? No lo sé; por esa razón se aplicarán tres filtros distintos y se compararán los resultados para encontrar el filtro que mejor se ajuste a nuestras necesidades.

Para aplicar los filtros, llamamos a los métodos correspondientes con parámetros convenientes.

```
img_ = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

gaussian = cv2.GaussianBlur(img_, (9, 9), 2)
median = cv2.medianBlur(img_, 9)
bilateral = cv2.bilateralFilter(img_, 9, 75, 70)

imagenes = [img, gaussian, median, bilateral]

titulos = ['Original', 'Gaussiano', 'Mediana', 'Bilateral']

plt.figure(figsize = (10, 10))

for i in range(len(imagenes)):

    plt.subplot(2, 2, i + 1)
    plt.imshow(imagenes[i], cmap = 'gray')
    plt.title(titulos[i])
    plt.xticks([]), plt.yticks([])

plt.tight_layout()
plt.show()
```

Listing 17: Filtro de imágenes.

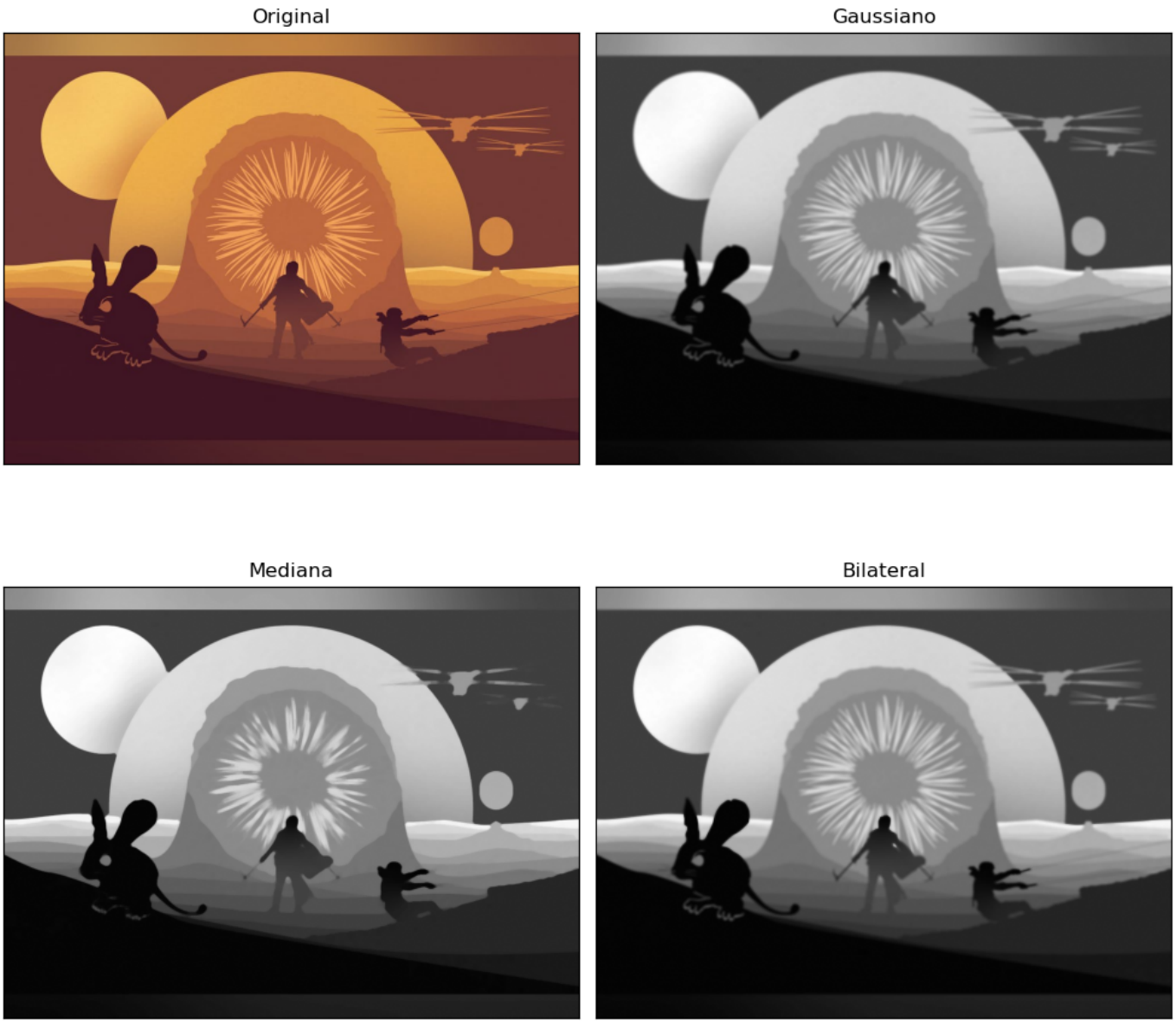

Figura 7: Comparación de los filtros y la imagen original.

4.2.3. Operadores Prewitt y Sobel para la detección de bordes

En esta sección nos encontramos con un problema similar a la pasada, ¿cómo podemos saber cuál operador nos conviene más?

En este caso compararemos los dos métodos y mostraremos las diferencias entre ellos.

```
sobelx = cv2.Sobel(gaussian, cv2.CV_64F, 1, 0, ksize = 3)
sobely = cv2.Sobel(gaussian, cv2.CV_64F, 0, 1, ksize = 3)
sobel_magnitud = np.sqrt(sobelx**2 + sobely**2)
sobel_magnitud = np.uint8(255 * sobel_magnitud / np.max(sobel_magnitud))
```

```python

kernelx = np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]])
kernely = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]])
prewittx = cv2.filter2D(gaussian, cv2.CV_64F, kernelx)
prewitty = cv2.filter2D(gaussian, cv2.CV_64F, kernely)
prewitt_magnitude = np.sqrt(prewittx**2 + prewitty**2)
prewitt_magnitude = np.uint8(255 * prewitt_magnitude / np.max(prewitt_magnitude))
```

Listing 18: Definición de operadores de Prewwit y Sobel.

Nota: En la última sección se compararán los resultados.

4.2.4. Transformada de Hough

Primero binarizariemos las imágenes para destacar los bordes. Después usaremos la biblioteca `skimage` para etiquetar las regiones; después, con `regionprops` obtendremos las propiedades de las regiones, las cuales ordenaremos, imprimiremos y contaremos para tener un poco más de información.

```python
# Binarizamos la imagen de bordes
_, umbralizada = cv2.threshold(sobel_magnitud, 50, 255, cv2.THRESH_BINARY)

# Etiquetamos las regiones conectadas
imagen_etiquetada = sk_label(umbralizada)

propiedades = regionprops(imagen_etiquetada)

print(f"Total de regiones: {len(propiedades)}.\n")

propiedades_ordenadas = sorted(propiedades, key = lambda p: p.area, reverse = True
    )

for i, prop in enumerate(propiedades_ordenadas[:5]):

    print(f"Región {i+1}:")
    print(f"  - Área: {prop.area} píxeles")
    print(f"  - Perímetro: {prop.perimeter:.2f}")
    print(f"  - Excentricidad: {prop.eccentricity:.2f}")
    print(f"  - Centro de masa (y, x): ({prop.centroid[0]:.2f}, {prop.centroid
    [1]:.2f})")
```

Listing 19: Obtención de regiones.

```
Total de regiones: 202.

Región 1:
  - Área: 12048.0 píxeles
  - Perímetro: 4586.89
  - Excentricidad: 0.79
  - Centro de masa (y, x): (244.00, 332.55)
Región 2:
  - Área: 4386.0 píxeles
```

```
  - Perímetro: 1789.76
  - Excentricidad: 0.76
  - Centro de masa (y, x): (322.29, 819.72)
Región 3:
  - Área: 3835.0 píxeles
  - Perímetro: 1499.18
  - Excentricidad: 1.00
  - Centro de masa (y, x): (41.52, 353.91)
Región 4:
  - Área: 2849.0 píxeles
  - Perímetro: 1085.42
  - Excentricidad: 0.56
  - Centro de masa (y, x): (447.42, 101.54)
Región 5:
  - Área: 2247.0 píxeles
  - Perímetro: 1021.08
  - Excentricidad: 0.73
  - Centro de masa (y, x): (445.40, 492.96)
```

Listing 20: Primeras cinco regiones.

Ahora, se definirá una función que detecte círculos mediante Hough.

```python
def detectar_circulo(imagen, titulo):

    if len(imagen.shape) == 3:

        img_gris = cv2.cvtColor(imagen, cv2.COLOR_RGB2GRAY)
        img_dibujo = imagen.copy()

    else:

        img_gris = imagen
        img_dibujo = cv2.cvtColor(imagen, cv2.COLOR_GRAY2RGB)

    circulos = cv2.HoughCircles(
        img_gris,
        cv2.HOUGH_GRADIENT,
        dp = 1,
        minDist = 25,
        param1 = 150,
        param2 = 30,
        minRadius = 0,
        maxRadius = 0
    )

    if circulos is not None:

        circulos = np.uint16(np.around(circulos))

        for c in circulos[0, :]:

            cv2.circle(img_dibujo, (c[0], c[1]), c[2], (0, 255, 0), 2)
            cv2.circle(img_dibujo, (c[0], c[1]), 2, (255, 0, 0), 3)
```

```

    plt.imshow(img_dibujo)
    plt.title(titulo)
    plt.xticks([]), plt.yticks([])
```

Listing 21: Función para detectar círculos.

Para finalizar esta sección, se preparará la visualización de los resultados comparando las diferencias de los operadores Prewitt y Sobel y se resaltarán para obtener una mejor visualización.

```
# Diferencia entre Sobel y Prewitt
diferencia = cv2.absdiff(sobel_magnitud, prewitt_magnitude)

diferencia_resaltada = cv2.normalize(diferencia, None, 0, 255, cv2.NORM_MINMAX)
```

Listing 22: .

4.2.5. Visualización y comparación de resultados

Primero se visualizarán los resultados obtenidos con Sobel y Prewitt y sus diferencias. Para apreciar mejor las diferencias, se optó por usar un `cmap = 'hot'`, entre más brille una región, más diferencias tendrán.

```
plt.figure(figsize = (15, 5))

plt.subplot(1, 3, 1)
plt.imshow(sobel_magnitud, cmap = 'gray')
plt.title('Bordes con Sobel')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(prewitt_magnitude, cmap = 'gray')
plt.title('Bordes con Prewitt')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(diferencia_resaltada, cmap = 'hot') # Los cambios brillan
plt.title('Diferencia Sobel y Prewitt')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Listing 23: Comparación de Prewitt y Sobel.

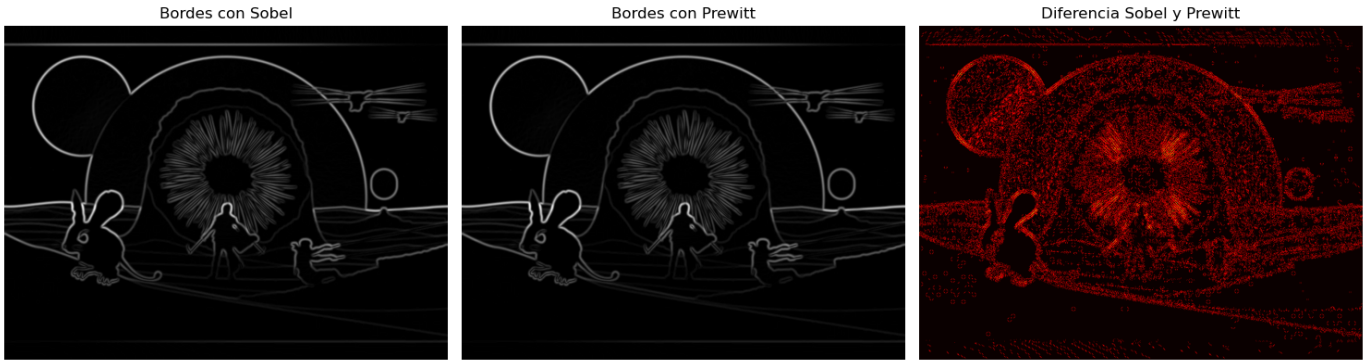

Figura 8: Comparación de operadores.

Para los resultados finales de los distintos filtros, compararemos las cuatro imágenes (original, filtro gaussiano, mediana y bilateral).

```
plt.figure(figsize = (12, 10))

for i in range(len(imagenes)):

    plt.subplot(2, 2, i + 1)
    detectar_circulo(imagenes[i], titulos[i])

plt.tight_layout()
plt.show()
```

Listing 24: Resultados con imagen original y con filtros.

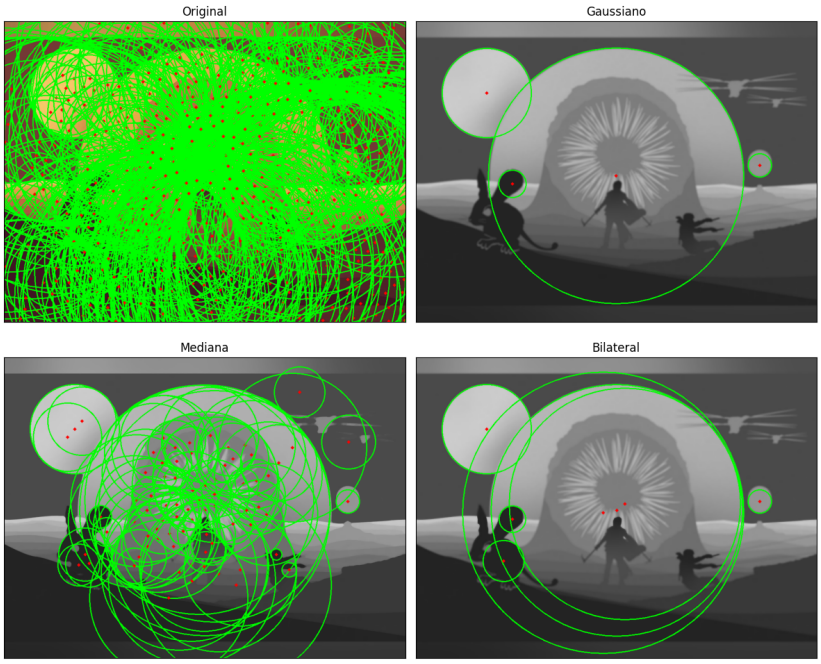

Figura 9: Resultados finales.

Y de las comparaciones, podemos concluir que, aunque hay muchas regiones diferentes aplicando Prewitt o Sobel, el resultado percibido nos es indiferente. Por otro lado, el resultado es bastante distinto al comparar los círculos generados con los distintos filtros y la imagen original; el resultado óptimo se obtiene con el filtro gaussian.

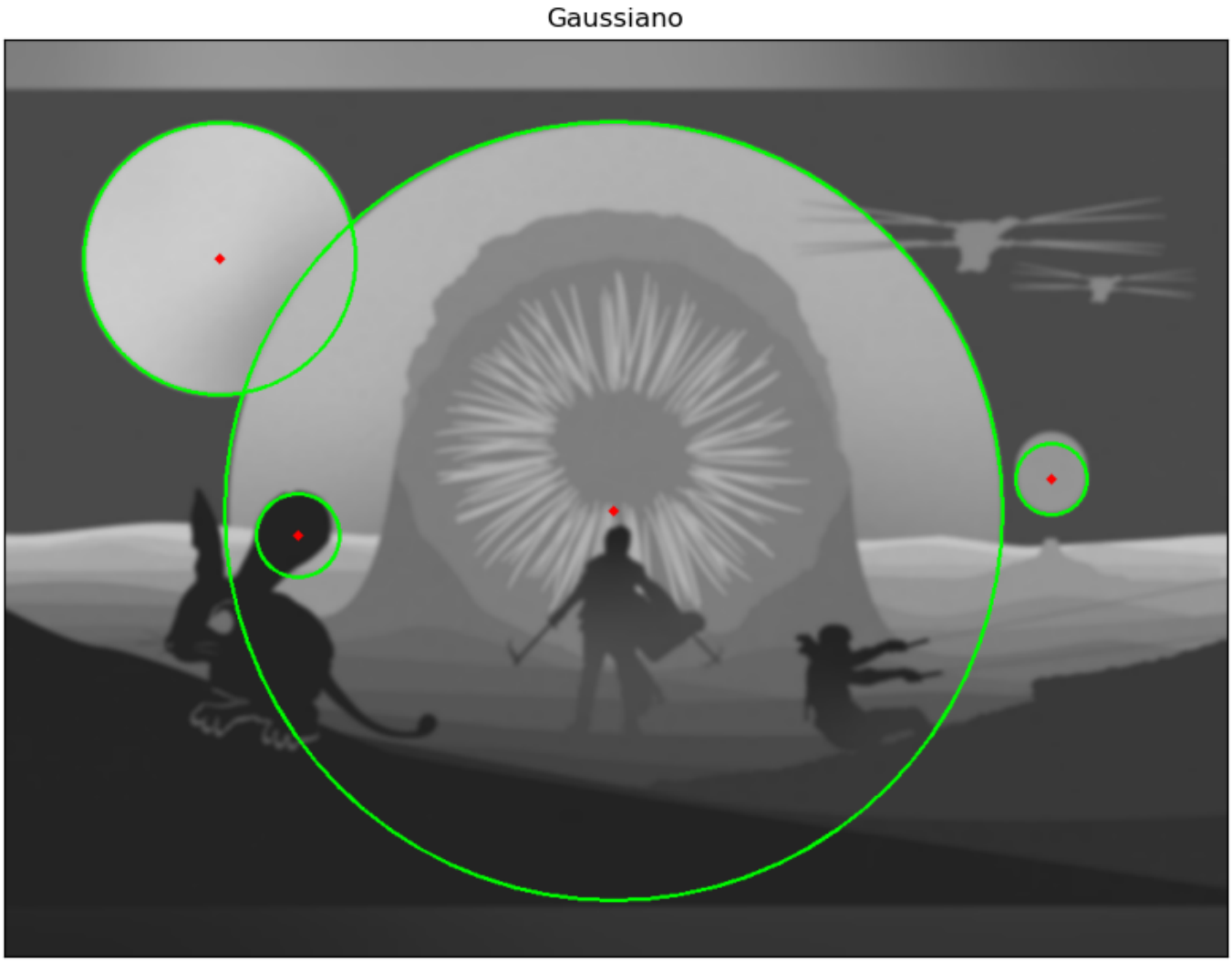

Figura 10: Resultado óptimo.

4.3. Señales sintéticas de ECG

Similar a los ejercicios anteriores, empezaremos importando las bibliotecas necesarias para trabajar con señales unidimensionales y aplicar los filtros correspondientes.

```
# Bibliotecas

import numpy as np
```

```
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt
```

Listing 25: Carga de bibliotecas para procesamiento de señales.

4.3.1. Generación de la señal original

Para crear nuestra señal sintética, utilizaremos la suma de funciones gaussianas. Cada función simulará una parte del complejo PQRST del latido cardíaco ajustando su amplitud, centro y ancho. Posteriormente, crearemos una señal continua sumando varios de estos latidos a lo largo de un vector de tiempo.

```
frecuencia = 500  # Frecuencia de muestreo
t = np.arange(0, 3, 1/frecuencia)  # Vector de tiempo de 3 segundos

# Utilizamos funciones Gaussianas para simular el PQRST
def simular_latido(t_local):
    ondas = [
        (0.15, -0.2, 0.02),   # Onda P
        (-0.15, -0.05, 0.01), # Onda Q
        (1.0, 0.0, 0.015),    # Onda R
        (-0.25, 0.05, 0.015), # Onda S
        (0.3, 0.3, 0.04)      # Onda T
    ]
    latido = np.zeros_like(t_local)
    for a, mu, sigma in ondas:
        latido += a * np.exp(-0.5 * ((t_local - mu) / sigma)**2)
    return latido

# Creamos una señal vacía y le sumamos latidos cada 1 segundo
ecg_original = np.zeros_like(t)
for i in range(4): # Latidos en los segundos 0, 1, 2 y 3
    ecg_original += simular_latido(t - i)
```

Listing 26: Generación de la señal sintética de ECG.

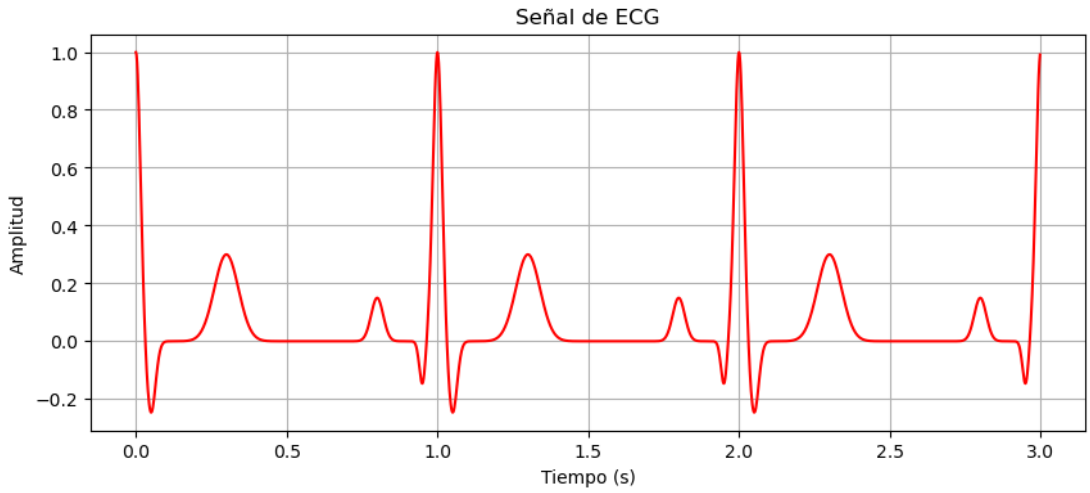

Figura 11: Señal sintética de ECG original.

4.3.2. Adición de ruido a la señal

Para emular un escenario realista y poner a prueba los filtros, agregaremos dos tipos de ruido a nuestra señal original: una interferencia eléctrica simulada con una onda senoidal de 60 Hz, y ruido aleatorio (ruido gaussiano) para emular la estática de los sensores.

```
ruido_60hz = 0.2 * np.sin(2 * np.pi * 60 * t)        # Interferencia electrica (60
    Hz)
ruido_gauss = np.random.normal(0, 0.08, len(t))      # Ruido aleatorio (Gaussiano)

ecg_ruidoso = ecg_original + ruido_60hz + ruido_gauss
```

Listing 27: Adición de ruido de 60 Hz y ruido gaussiano.

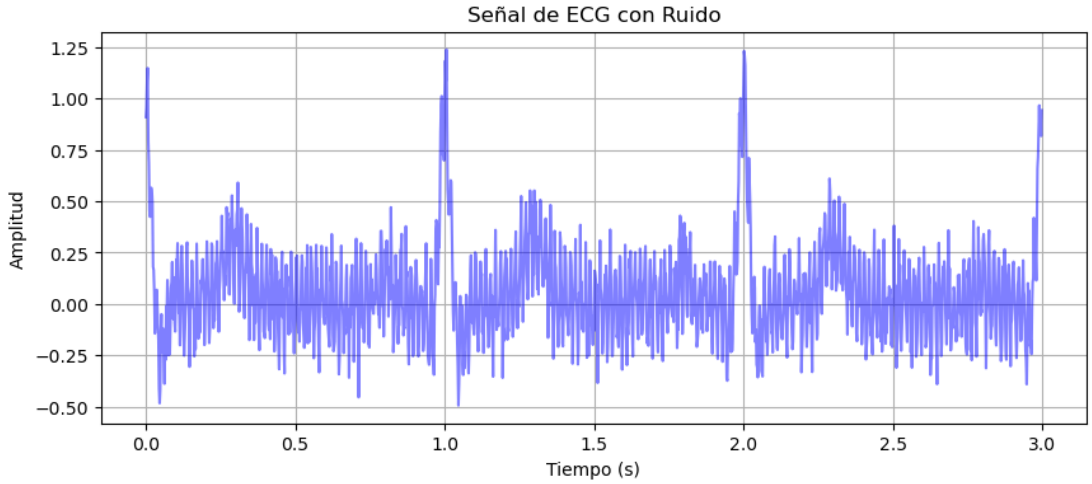

Figura 12: Señal de ECG con ruido.

4.3.3. Transformada de Fourier y filtrado en frecuencia

Para analizar las componentes de frecuencia de la señal contaminada, utilizaremos la Transformada Rápida de Fourier (FFT). Posteriormente, diseñaremos un filtro ideal en el dominio de la frecuencia estableciendo a cero todas las frecuencias superiores a 40 Hz, y regresaremos al dominio del tiempo con la transformada inversa (IFFT).

```
N = len(t)
freqs = np.fft.fftfreq(N, 1/frecuencia)
fft_ruidoso = np.fft.fft(ecg_ruidoso)

# Hacemos una copia y establecemos a cero las frecuencias mayores a 40 Hz
fft_filtrado = fft_ruidoso.copy()
fft_filtrado[np.abs(freqs) > 40] = 0

# Transformada inversa (IFFT) para volver al dominio del tiempo
ecg_filtrado_freq = np.real(np.fft.ifft(fft_filtrado))
```

Listing 28: Aplicación de la FFT y filtrado en el dominio de la frecuencia.

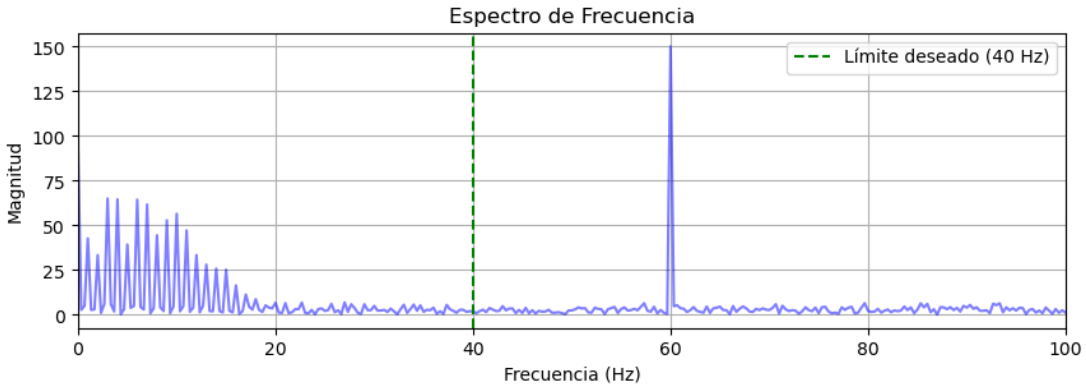

Figura 13: Espectro de frecuencia de la señal con ruido.

4.3.4. Filtrado en el dominio del tiempo

De manera opcional y para tener un punto de comparación, se diseñó un filtro pasabajas de Butterworth de orden 4 con una frecuencia de corte de 40 Hz. Para evitar el desfasamiento temporal de la señal, se aplicó el filtro utilizando la función `filtfilt`.

```
# Filtro Butterworth pasa bajas de orden 4, frecuencia de corte 40 Hz
b, a = butter(4, 40, fs=frecuencia, btype='low')

# Usamos filtfilt para no generar desfasaje temporal
ecg_filtrado_tiempo = filtfilt(b, a, ecg_ruidoso)
```

Listing 29: Filtro Butterworth pasabajas en el dominio del tiempo.

4.3.5. Visualización y comparación de resultados

Para finalizar esta sección, se preparará la visualización de los resultados superponiendo la señal original, la señal con ruido y las dos señales filtradas para observar la efectividad de los métodos aplicados.

```
plt.figure(figsize = (12, 6))

plt.plot(t, ecg_ruidoso, label = 'Señal con Ruido', color = 'gray', alpha = 0.3)
plt.plot(t, ecg_original, label = 'Original', color = 'green', linewidth = 1.5,
    alpha = 0.8)
plt.plot(t, ecg_filtrado_freq, label = 'Filtrada (Frecuencia)', color = 'red',
    linestyle = 'dashed', linewidth = 2, alpha = 0.6)
plt.plot(t, ecg_filtrado_tiempo, label = 'Filtrada (Tiempo)', color = 'blue',
    linestyle = 'dotted', linewidth = 2)

plt.title('Comparación y Superposición de Señales')
plt.xlabel('Tiempo (s)')
plt.ylabel('Amplitud')
plt.xlim(0, 3)
plt.legend(loc = 'upper right')
plt.grid(True, alpha = 0.3, linestyle = '--')
plt.show()
```

Listing 30: Comparación y superposición de las señales de ECG.

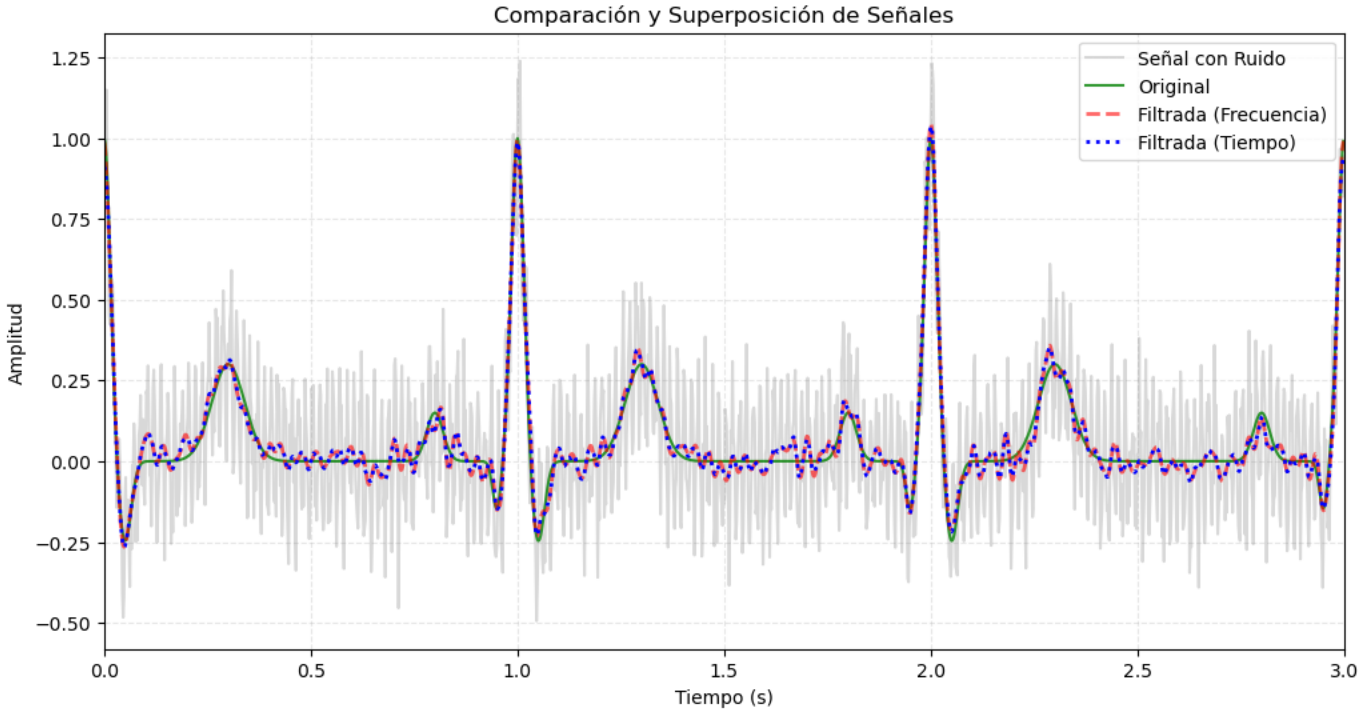

Figura 14: Comparación de la efectividad de los filtros en la señal de ECG.

5. Conclusiones

A lo largo de esta práctica, se logró observar de manera experimental la importancia crítica del preprocesamiento y limpieza de datos, imágenes y señales dentro de las tareas de Reconocimiento de Patrones.

En la primera sección, el análisis exploratorio y de correlación del conjunto de datos Iris nos permitió identificar dependencias positivas fuertes entre las medidas morfológicas (como la longitud y ancho de los pétalos), lo cual es fundamental para el posterior entrenamiento de modelos de clasificación geométrica.

En la segunda sección, comprobamos que la aplicación de filtros de suavizado (destacando el filtro Gaussiano como el resultado óptimo) es un paso indispensable antes de aplicar técnicas de extracción de características como el operador de Sobel/Prewitt o la Transformada de Hough. El ruido en la imagen original generaba falsos positivos masivos en la detección de bordes y círculos, los cuales fueron mitigados eficazmente mediante el filtrado.

Finalmente, en el procesamiento de señales de ECG, pudimos comprobar la efectividad matemática de las transformaciones de datos. La señal ruidosa ocultaba casi por completo las ondas de baja amplitud (P y T) del latido. Sin embargo, demostramos que tanto el truncamiento espectral en el dominio de la frecuencia (vía FFT/IFFT) como el uso de un filtro IIR en el dominio del tiempo (Butterworth) son altamente efectivos para atenuar la interferencia de alta frecuencia y recuperar la integridad de la morfología original de la señal cardiovascular.

■